```
  puts "#{p1.name}'s name > #{p2.name}'s name"
end

puts "Sorted list:"
puts [p1, p2, p3].sort
```

produces:

```
Matz's name > Guido's name
Sorted list:
Guido
Larry
Matz
```

We included Comparable in our Person class and then defined a <=> method. We were then able to perform comparisons (such as p1 > p2) and even sort an array of Person objects.

Inheritance and Mixins

Some object-oriented languages (such as C++ or Python) support multiple inheritance, where a class can have more than one immediate parent, inheriting functionality from each. Although powerful, this technique can be dangerous, because the inheritance hierarchy can become ambiguous.

Other languages, such as Java, JavaScript, and C#, support single inheritance. Here, a class can have only one immediate parent. Although cleaner (and easier to implement), single inheritance also has drawbacks—in the real world, objects often inherit attributes from multiple sources (a ball is both a *bouncing thing* and a *spherical thing*, for example).

Ruby offers an interesting and powerful compromise, giving you the simplicity of single inheritance and the power of multiple inheritance. A Ruby class has only one direct parent, so Ruby is a single-inheritance language. However, Ruby classes can include the functionality of any number of *mixins* (a mixin is like a partial class definition). This provides a controlled multiple-inheritance-like capability with none of the drawbacks.

Ruby provides two mechanisms for mixing in module behavior which are related to include but combine the module and the class differently. The behavior of include is to add the module's methods as instance methods to the class in which the module is being included, and to have those module methods be looked up after the class itself is checked for a method.

Ruby also provides extend. The behavior of extend is to add the methods directly to the receiver of extend rather than as instance methods of a class. As most commonly used, the effect of extend is to add the module methods as class methods:

```
module ExtendedNew
  def new_from_string(string, delimiter = ",")
    new(*string.split(delimiter))
  end
end

class Person
  extend ExtendedNew

  def initialize(first_name, last_name)
    @first_name = first_name
    @last_name = last_name
  end

  def full_name = "#{@first_name} #{@last_name}"
```